

Software Design Document

CultiKure: An AI-Powered Web Application for Plant Disease Detection and Cure Recommendation

Version: 1.0



Submitted by

Abdullah Javeed (F22BINFT1M01076)

Submitted to

DR. Syed Ali Nawaz Shah

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Summary

This Software Design Document (SDD) explains the design of the CultiKure web application. The main purpose of this document is to describe how the system will be designed and structured in order to implement the requirements defined in the Software Requirements Specification (SRS).

CultiKure is a web-based application that helps users identify plant diseases and provides suitable cure recommendations. The system is designed using an API-first approach, where the frontend interacts with a backend server developed using Flask. The backend handles user requests, controls the system workflow, and manages communication with external AI services.

The application uses the Plant.id API to analyze uploaded plant images and detect diseases. After detection, the Google Gemini API is used to generate detailed information about the disease along with possible cure recommendations. User information and scan history are stored in a lightweight database during development, with the option to use a cloud-based database for future scalability.

This document focuses on explaining the overall application architecture, major system modules, database design, and user interface structure. The design aims to keep the system simple, modular, and easy to maintain so that it can be extended in the future and easily used by farmers and gardeners with limited technical knowledge.

Table of Contents

1. Application Architecture	4
1.1 MVC Design Approach	4
1.2 Controllers and Their Functions	4
1.3 External APIs and Libraries	5
1.4 UML Diagrams and Design Representation	5
1.5 Coding Conventions	7
2. Data Model Schema	8
2.1 Entity 1: User	8
2.2 Entity 2: ScanHistory	9
2.3 Entity 3: Entity Relationship	10
3. User Interface	9
3.1. Overall Design Philosophy	11
3.2. List of Screens and Functionality	12

1. Application Architecture

The CultiKure web application is designed using a simple and modular architecture to ensure clarity, scalability, and ease of maintenance. The system follows an **API-first and MVC-based approach**, where responsibilities are clearly divided between the user interface, backend logic, and external AI services.

At a high level, the application is divided into three main layers: the frontend layer, the backend layer, and the external services layer.

The **frontend layer** is responsible for user interaction. It is developed using HTML, CSS, and JavaScript and runs directly in the user's web browser. This layer allows users to register, log in, upload plant images, view diagnosis results, and access scan history. The frontend communicates with the backend using HTTP requests.

The **backend layer** acts as the core controller of the system and is developed using the Flask framework in Python. This layer follows the **Controller part of the MVC pattern** and handles application logic, request validation, session management, and communication with external APIs. The backend does not directly expose internal processing details to the user and works as a coordination layer.

1.1 MVC Design Approach

Although the system does not strictly separate all components into full MVC layers, the overall design follows MVC principles:

- **Models** represent the data structures such as users and scan history records stored in the database.
- **Views** represent the frontend web pages rendered in the browser.
- **Controllers** handle incoming requests and manage the flow of data between views, models, and external APIs.

As per the design guidelines, only the controller responsibilities are explained below.

1.2 Controllers and Their Functions

The backend contains the following main controllers:

1. **Authentication Controller**
 - Handles user registration and login.
 - Validates user credentials.
 - Manages user sessions.
2. **Image Upload Controller**
 - Receives plant image files from the frontend.
 - Validates file format and size.

- Forwards valid images to the disease detection service.
- 3. **Disease Detection Controller**
 - Communicates with the Plant.id API.
 - Sends image data for analysis.
 - Receives disease detection results.
- 4. **Cure Recommendation Controller**
 - Sends disease information to the Google Gemini API.
 - Receives detailed disease descriptions and cure recommendations.
 - Prepares results for display.
- 5. **History Management Controller**
 - Stores scan results in the database.
 - Retrieves past scan records for logged-in users.

1.3 External APIs and Libraries

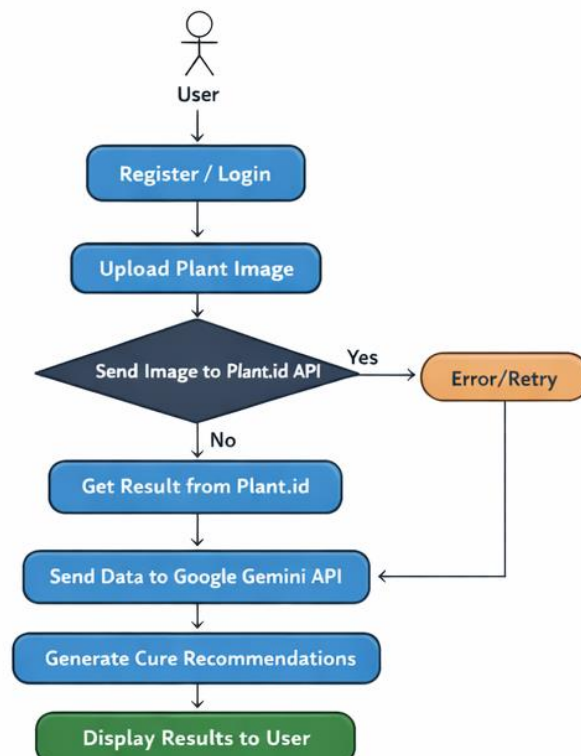
The system relies on external AI services to perform its core functionality:

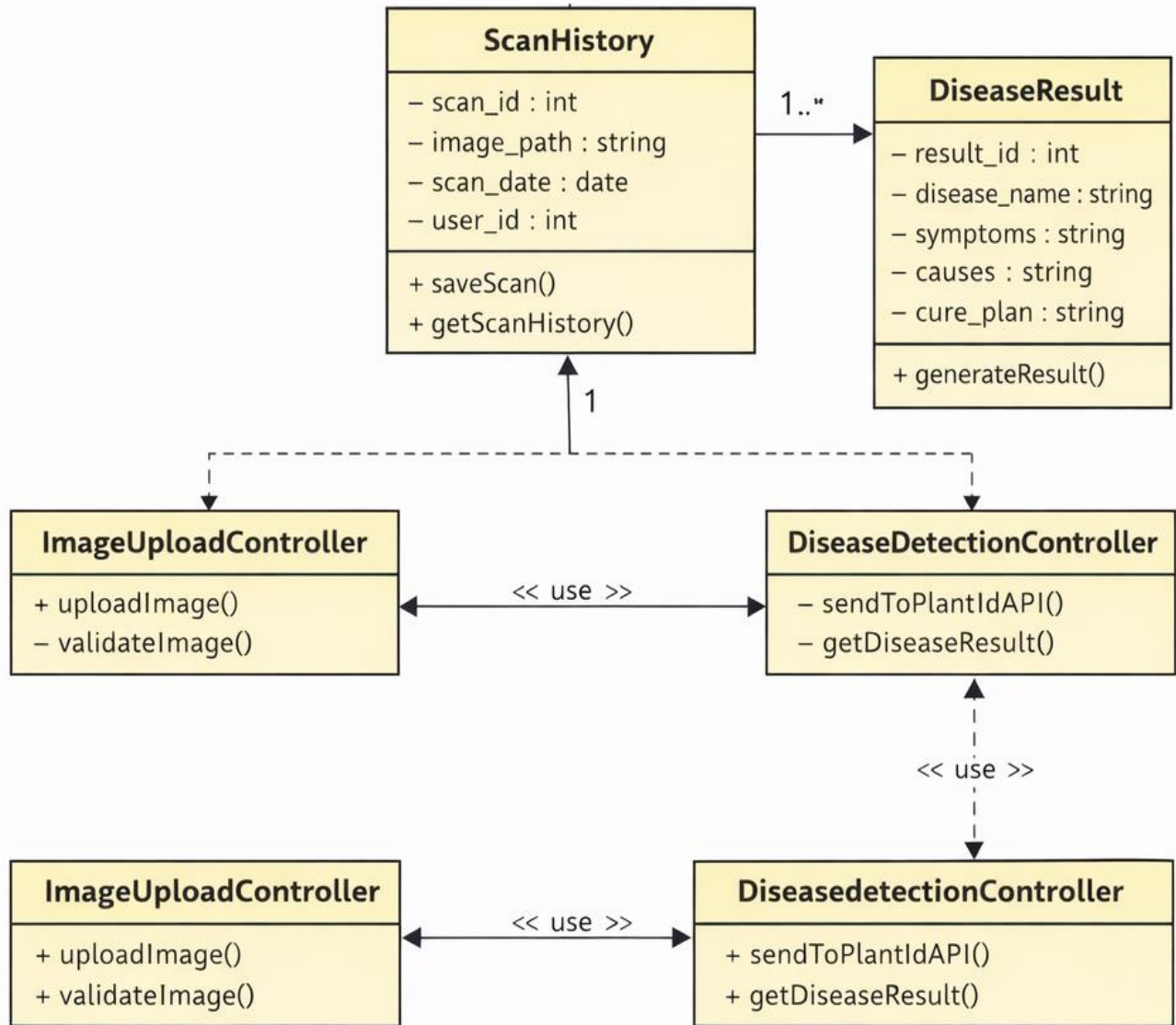
- The **Plant.id API** is used to analyze uploaded plant images and detect diseases.
- The **Google Gemini API** is used to generate detailed disease information and cure recommendations based on the detection results.

These APIs are accessed securely through the backend, and their keys are stored using environment variables to ensure security.

1.4 UML Diagrams and Design Representation

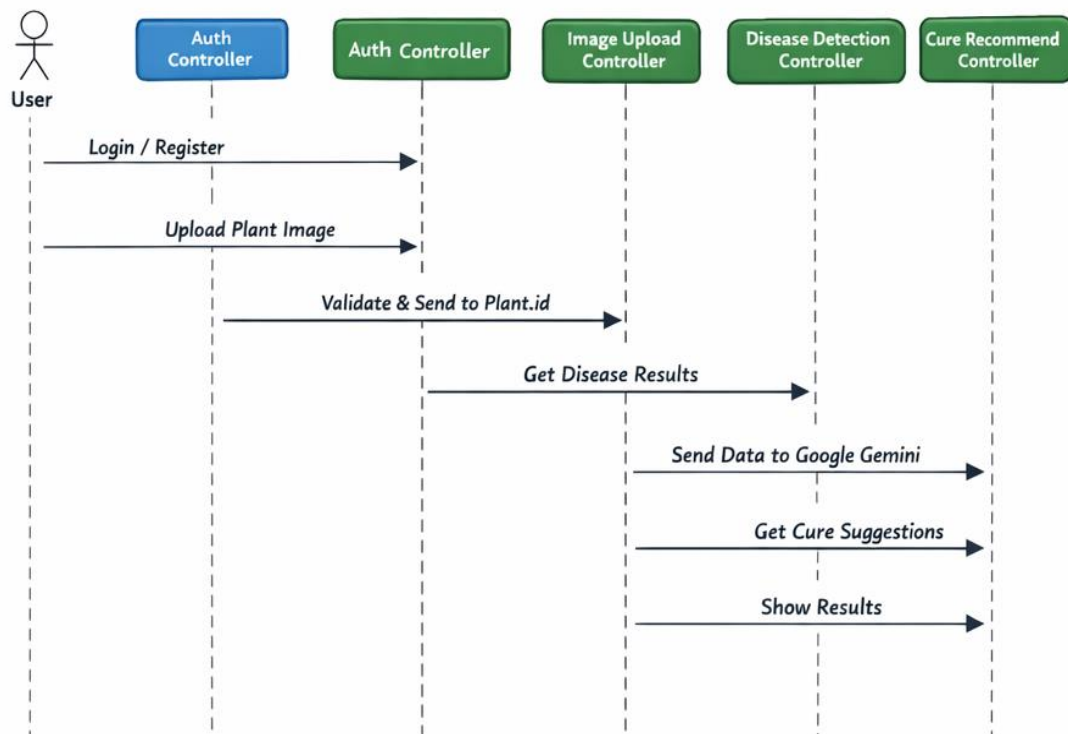
System Activity Diagram





Class Diagram – Cultikure

Controllers Interaction Sequence



1.5 Coding Conventions

The coding conventions followed in the development of the **CultiKure** web application are aimed at maintaining clarity, consistency, and ease of maintenance. Standard practices are used throughout the project to ensure that the code remains readable and understandable for future development.

The **backend** of the application is developed using **Python** and the **Flask** framework.

Descriptive and meaningful names are used for **variables**, **functions**, and **controllers** so that their purpose can be easily understood. Function names follow a clear naming pattern and are written in lowercase with words separated by underscores.

The application logic is organized into separate controllers based on functionality, such as **authentication**, **image upload**, **disease detection**, **cure recommendation**, and **history**

management. Each controller handles a specific **responsibility**, which helps in keeping the code modular and easy to manage.

Proper indentation and formatting are maintained according to standard Python guidelines.

Comments are added where necessary to explain important logic or complex operations.

Reusable code is preferred to avoid duplication and improve maintainability.

Sensitive information such as **API keys** and **configuration** details are not hard-coded in the source files. Instead, they are stored securely using environment variables to improve security.

Basic error handling is implemented to manage invalid inputs, API failures, and unexpected system behavior in a controlled manner.

Overall, these coding conventions ensure that the application code is structured, secure, and easy to extend in the future.

2. Data Model Schema

2.1 Entity 1: User

The **User** entity represents registered users of the system, including farmers and gardeners.

Attributes:

- **user_id** (Primary Key): A unique identifier for each user
- **name**: Full name of the user
- **email**: Email address used for login and identification
- **password**: Encrypted password for authentication

Description:

Each user must register before accessing the system's core features. The user entity is responsible for maintaining authentication details and serves as the parent entity for scan history records.

2.2 Entity 2: ScanHistory

The **ScanHistory** entity stores details of plant disease scans performed by users.

Attributes:

- **scan_id** (Primary Key): Unique identifier for each scan
- **image_path**: Path or reference to the uploaded plant image

- `scan_date`: Date when the scan was performed
- `disease_name`: Name of the detected disease
- `cure_recommendation`: Detailed symptoms and cure advice generated by AI
- `user_id` (Foreign Key): References the associated user

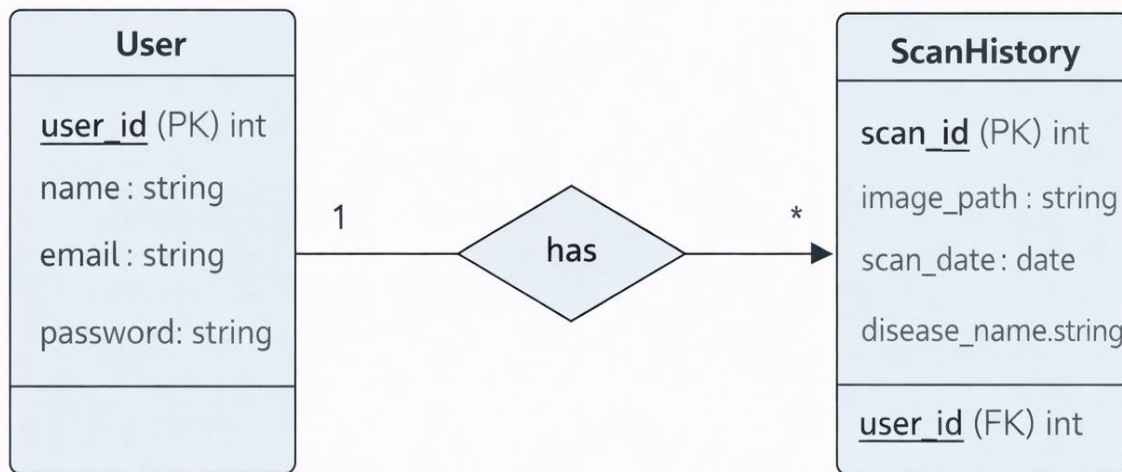
Description:

Each scan record is linked to a specific user. This entity allows the system to maintain a history of plant disease detections so that users can review previous results at any time.

2.3 Entity Relationships

- A **User** can have **multiple ScanHistory records**
- Each **ScanHistory** record belongs to **one User**

This represents a **one-to-many (1:N) relationship** between User and ScanHistory.



ER Diagram – CultiKure

3. User Interface

3.1. Overall Design Philosophy: The User Interface (UI) of CultiKure adheres to a **User-Centric Design** philosophy, focusing on accessibility and ease of use for the agricultural community.

- **Color Scheme:** The application adopts a **high-contrast, professional color palette**. The primary focus is on visual clarity to ensure that text and data are readable under various lighting conditions (e.g., outdoor field usage).
- **Typography:** The system utilizes **standard modern Sans-Serif fonts** (System Fonts). This ensures rapid loading times and native legibility across different operating systems and browsers.
- **Design Niche:** The interface follows a **Minimalist** design niche. Clutter is minimized to highlight the core functionality, disease detection, reducing the cognitive load on the user.
- **Responsiveness:** The application is built on **Responsive Web Design (RWD)** principles. It dynamically adjusts layout containers, images, and font sizes to fit targeted screen sizes, ranging from compact mobile viewports to full desktop displays.

3.2. List of Screens and Functionality: The application workflow is divided into the following functional screens:

A. Registration Screen

- **Purpose:** Enables new users to register and create a secure identity within the system .
- **Data Displayed:** Data entry forms for personal profile creation.
- **Controls & Working:**
 - Input Fields (Text): For capturing User Name and Email Address.
 - Input Fields (Password): For capturing and confirming secure credentials.
 - Primary Button (Register): Validates the input format and submits the data to the server to create a new record.
- **Input Constraints:**
 - Standard email format validation.
 - Password length and complexity checks (min 8 characters).

B. Login Screen

- **Purpose:** Authenticates users to provide access to protected features.
- **Data Displayed:** Authentication entry points.
- **Controls & Working:**
 - Input Fields: For Email and Password entry.
 - Primary Button (Login): Triggers the authentication protocol; successful validation redirects to the Dashboard.

- **Input Constraints:** Mandatory fields; system prevents submission of empty forms.

C. Main Dashboard

- **Purpose:** The central workspace for initiating scans and navigation.
- **Data Displayed:** System status, navigation menu, and upload interface.
- **Controls & Working:**
 - Action Button (Upload/Capture): Interacts with the device's operating system to open the File Picker or Camera.
 - Navigation Links: To access "Scan History" or "Profile Settings".
 - Button (Logout): Securely ends the user session.
- **Input Constraints:**
 - File Type: Restricted to standard image formats (JPG, PNG).
 - File Size: Restricted to optimized limits (e.g., 5MB) to ensure server performance.

D. Diagnosis Result Screen

- **Purpose:** Presents the AI-generated analysis and actionable advice.
- **Data Displayed:**
 - **Identification:** The detected disease name.
 - **Analysis:** Confidence score and detailed symptoms.
 - **Prescription:** Structured cure recommendations (Organic & Chemical).
- **Controls & Working:**
 - Action Button: Options to "Rescan" or "Save Report".
- **Input Constraints:** Read-only view for the user.

E. Scan History Screen

- **Purpose:** Provides a historical log of user activities.
- **Data Displayed:** Chronological list or grid of past diagnostic cards.
- **Controls & Working:**
 - Interactive List Items: Clicking a record retrieves and displays the full details of that specific scan from the database.
- **Input Constraints:** None (Read-Only Interface). The system prevents modification of historical records to maintain data integrity.